

TP6 Derived Types

Zyad Fri

March 2026

Exercise 1 Matrix Transposition using Derived Types

Implementation

A 4×5 matrix `a` is initialized on process 0 with values 1 to 20. The goal is for process 1 to receive the transposed 5×4 matrix directly in memory using MPI derived types — no manual transposition.

Two derived types are built on the sender side (process 0):

- `MPI_Type_vector(4, 1, 5, MPI_DOUBLE, &col_type)` — describes one column: 4 elements each separated by a stride of 5 (the row width).
- `MPI_Type_create_hvector(5, 1, sizeof(double), col_type, &trans_type)` — repeats that column 5 times, each starting 8 bytes (1 double) further. This sends the data column-by-column.

Process 0 calls `MPI_Send(a, 1, trans_type, 1, 0, ...)` and process 1 receives 20 raw doubles into `at[5][4]`. Because data arrives column-by-column, it fills the transposed array row-by-row.

Output

```
1 Process 0 - Matrix a:
2 1  2  3  4  5
3 6  7  8  9 10
4 11 12 13 14 15
5 16 17 18 19 20
6
7 Process 1 - Matrix transposee at:
8 1  6  11 16
9 2  7  12 17
10 3  8  13 18
11 4  9  14 19
12 5 10  15 20
```

The output matches the expected result. The transposition is performed entirely through the derived type during communication — no extra loop or temporary buffer is used.

Exercise 2 - Distributed Gradient Descent with MPI Derived Types

Implementation

A custom MPI struct type is defined for the `Sample` struct using `MPI_Type_create_struct`, with `MPI_Get_address` to compute the exact byte offsets of the feature array and the label. The dataset (1000 samples, 5 features) is generated on process 0 and scattered with `MPI_Scatter`. Each process computes its local gradient and MSE, then `MPI_Allreduce` aggregates both. All processes update weights identically every epoch. Early stopping triggers when $\text{MSE} < 10^{-2}$.

Selected Output (2 processes)

```
1 Epoch 10 | Loss (MSE): 5.629772 | w[0]: 0.0591, w[1]: -0.0278
2 Epoch 100 | Loss (MSE): 3.118820 | w[0]: 0.5195, w[1]: -0.2468
3 Epoch 500 | Loss (MSE): 0.230076 | w[0]: 1.5623, w[1]: -0.7636
4 Epoch 900 | Loss (MSE): 0.018053 | w[0]: 1.8734, w[1]: -0.9285
5 Epoch 990 | Loss (MSE): 0.010480 | w[0]: 1.9045, w[1]: -0.9457
6 Early stopping at epoch 998 - loss 0.009996 < 1.0e-02
7 Training time: 0.012 seconds (MPI)
```

Convergence at epoch 998. Weights converge toward the true values $\mathbf{w_true} = [2.0, -1.0, 3.0, 0.5, -2.0]$.

MSE Convergence Curve

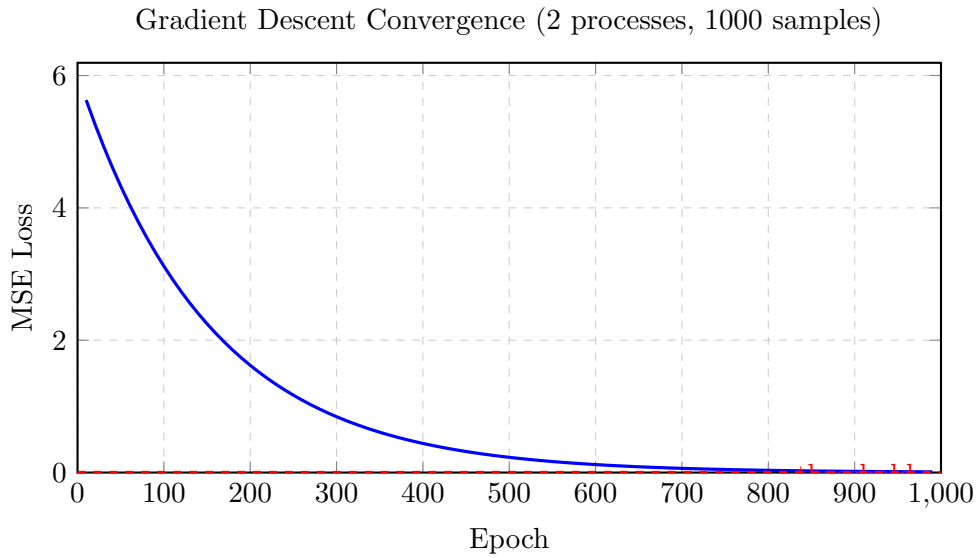


Figure 1: MSE decreases steadily and crosses the stopping threshold at epoch 998.

Speedup and Efficiency (Local Measurements)

Measured on local machine with `N_SAMPLES = 1000`:

Processes	Time (s)	Speedup $S_p = T_1/T_p$	Efficiency $E_p = S_p/p$
1	0.009	1.00	1.00
2	0.012	0.75	0.38
4	0.014	0.64	0.16

Table 1: Timing results on local machine (small dataset).

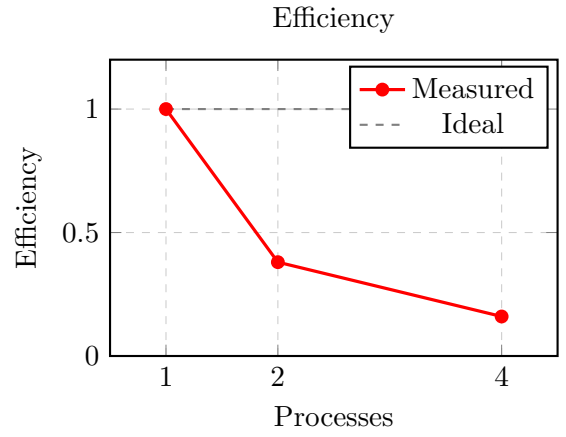
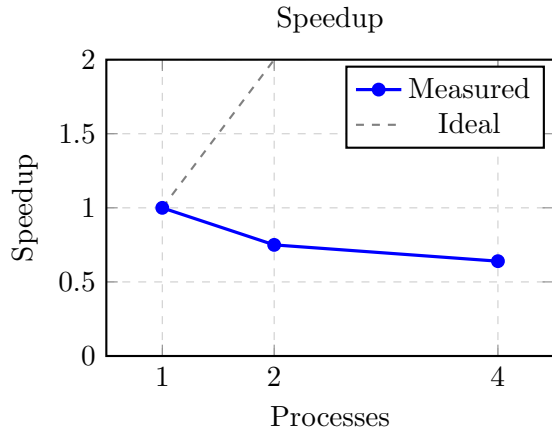


Figure 2: With only 1000 samples, `MPI_Allreduce` overhead per epoch dominates compute time, giving sub-linear speedup. On Toubkal with 10^6+ samples, compute dominates and near-linear speedup is expected up to a saturation point.